Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра вычислительных систем

ОТЧЕТ
по практической работе 5

по дисциплине «**Основы разработки на Java**»

Выполнил:
студент гр. ИС-542
«11» октября 2026 г. _____ /Кузугашев С.В./

Проверил:
Ассистент кафедры ВС
«12» октября 2026 г. _____ /Гузенков С.К./

Оценка «_____»

Новосибирск 2026

Задание

Дисциплина «Программирование». Лабораторная работа №3. Тема: полиморфизм и проектирование на Java — динамическая диспетчеризация, интерфейсы и абстрактные классы, композиция, sealed-иерархии и расширяемый движок правил. Среда: OpenJDK 21.
Исходный код лабораторной работы опубликован в открытом репозитории: https://github.com/Forlein/SibGUTI-Java-Labs (папка WORK_Java/laboratory_work_3).
В работе реализованы 15 заданий:
1. Один контракт, две реализации — интерфейс CourseFormatter и две реализации (Compact, Detailed).
2. Dynamic dispatch на бумаге — иерархия NotificationSender и разбор выбора перегрузки/переопределения.
3. Полиморфный массив — массив интерфейсного типа и обход циклом for-each.
4. Абстрактный базовый класс — final-алгоритм send и точка расширения doSend.
5. Интерфейс или абстрактный класс — выбор типа для Identifiable, BaseEntity, Formatter.
6. Переопределение без случайной перегрузки — роль @Override при смене типа параметра.
7. Композиция вместо наследования — декораторы LoggingSender и RetrySender.
8. Нарушение LSP — разделение CourseReader и CourseWriter.
9. Политика допуска как стратегия — интерфейс EnrollmentPolicy и три реализации.
10. Конфликт default-методов — разрешение через Interface.super.method().
11. Равенство в иерархии — симметрия equals для Point и ColoredPoint.
12. Sealed-иерархия результата — исчерпывающий switch без default.
13. Удаление type code — замена switch по строке на полиморфизм.
14. Наследование взрывает комбинации — две независимые оси Format и Payment.
15. Расширяемый движок правил — sealed RuleResult, Rule, AllOf и AnyOf.
Работа выполнена на оценку «5»: реализованы все 15 заданий, проект компилируется без ошибок, каждая программа запускается и выдаёт корректный результат.

Выполнение работы

Ниже по каждому заданию приведена суть выполненной работы и краткий ответ для защиты.

Задача 1. Один контракт, две реализации

Что сделали: Создали интерфейс CourseFormatter с методом format(Course). Написали две реализации (Compact и Detailed). Вызвали их через переменную типа интерфейса.

Ответ для защиты: Компилятору известен только тип переменной (CourseFormatter). Он проверяет, что у этого типа есть метод format(), и что вызов синтаксически корректен. Но компилятор не знает, какой конкретно объект (Compact или Detailed) лежит в переменной. JVM во время выполнения смотрит на фактический объект и вызывает именно его реализацию. Это и есть динамическая диспетчеризация.

Задача 2. Dynamic dispatch на бумаге

Что сделали: Создали иерархию NotificationSender → EmailSender, ConsoleSender. Добавили перегруженный метод send(Object) и send(String). Предсказали вывод.

Ответ для защиты: В Java выбор метода идёт в два этапа. Overload (перегрузка) решается компилятором по типу ссылки и аргументов. Override (переопределение) решается JVM по фактическому типу объекта. Если наследник переопределил только send(String), то вызов send((Object)"...") выберет сигнатуру Object, но так как она не переопределена в наследнике, сработает метод родителя.

Задача 3. Полиморфный массив

Что сделали: Сложили несколько CourseFormatter в массив типа интерфейса. Прогнали их циклом for-each. Добавили третью реализацию (Json), не меняя цикл.

Ответ для защиты: Клиент (цикл) зависит только от интерфейса. Он не содержит if или instanceof. При добавлении новой реализации мы просто добавляем новый элемент в массив. Цикл не меняется. Это демонстрирует принцип «открыт для расширения, закрыт для изменения».

Задача 4. Абстрактный базовый класс

Что сделали: Создали AbstractNotificationSender с общей проверкой адресата (метод send помечен final) и абстрактным методом doSend. Наследники реализуют только doSend.

Ответ для защиты: Публичный алгоритм send() помечен final, чтобы наследники не могли его переопределить и случайно удалить проверку адресата. Точка расширения — абстрактный метод doSend(). Наследники переопределяют только его, не дублируя общую логику.

Задача 5. Интерфейс или абстрактный класс

Что сделали: Выбрали типы для понятий Identifiable, CoursePolicy, BaseEntity, Formatter, ClockProvider.

Ответ для защиты: Identifiable и Formatter — интерфейсы (чистые контракты без состояния). BaseEntity — абстрактный класс (нужно общее поле id и код для equals/hashCode). CoursePolicy — интерфейс-стратегия (позволяет добавлять новые правила без изменения сервиса). ClockProvider — функциональный интерфейс (для подмены времени в тестах). Главный критерий выбора: если нужно хранить состояние (поля) — абстрактный класс; если только поведение — интерфейс.

Задача 6. Переопределение без случайной перегрузки

Что сделали: Исправили ошибки в параметрах и доступе. Показали, что без @Override код компилируется, но вызывается не тот метод (из-за изменения параметра с Course на Object).

Ответ для защиты: Без @Override компилятор не проверяет, что метод реально переопределяет родительский. Если изменить тип параметра, это становится перегрузкой, и вызывается не тот метод. @Override заставляет компилятор проверить сигнатуру и выдать ошибку ещё до запуска.

Задача 7. Композиция вместо наследования

Что сделали: Создали декораторы LoggingSender и RetrySender, реализующие NotificationSender и принимающие его в конструктор. Показали два порядка композиции.

Ответ для защиты: Вместо наследования мы используем декораторы, оборачивающие друг друга. Если LoggingSender снаружи, а RetrySender внутри — логирование происходит один раз. Если наоборот — каждая попытка повтора логируется отдельно. Композиция позволяет комбинировать поведение в любом порядке без создания классов-мутантов.

Задача 8. Нарушение LSP

Что сделали: Разделили интерфейс CourseRepository на CourseReader (только чтение) и CourseWriter (чтение + запись). ReadOnly реализует только CourseReader.

Ответ для защиты: Наследник не должен ломать контракт родителя. Если ReadOnlyRepository не умеет сохранять, он не должен обещать это через интерфейс. Разделение интерфейсов позволяет read-only реализации честно реализовывать только чтение, не выбрасывая исключений.

Задача 9. Политика допуска как стратегия

Что сделали: Создали интерфейс EnrollmentPolicy и три реализации. EnrollmentService принимает политику через конструктор.

Ответ для защиты: Использован паттерн «Стратегия». Сервис не знает о конкретных политиках, он просто вызывает isAllowed(). Добавление новой политики не требует изменения сервиса. Политика внедряется через конструктор, что делает код гибким и тестируемым.

Задача 10. Конфликт default-методов

Что сделали: Создали два интерфейса с одинаковым default-методом. Получили ошибку компиляции. Разрешили конфликт через @Override и явный вызов Interface.super.method().

Ответ для защиты: При множественном наследовании default-методов возникает неоднозначность. Java не угадывает, чей метод использовать, а требует явного выбора через переопределение. Это защищает от скрытых багов.

Задача 11. Равенство в иерархии

Что сделали: Воспроизвели нарушение симметричности equals между Point и ColoredPoint. Предложили два решения: строгое сравнение по классу и композиция цвета.

Ответ для защиты: Если наследник добавляет поля в equals, симметрия ломается (p.equals(cp) может быть true, а cp.equals(p) — false). Строгое сравнение по классу сохраняет симметрию, но нарушает LSP. Композиция (класс ColoredPoint содержит Point внутри) решает проблему кардинально.

Задача 12. Sealed-иерархия результата

Что сделали: Создали sealed interface EnrollmentResult с тремя реализациями. Обработали все варианты через switch без default.

Ответ для защиты: sealed ограничивает круг наследников. Компилятор знает все варианты и требует их обработки в switch. При добавлении нового типа компилятор заставит обновить все switch в коде. Это защищает от ошибок, когда новый тип не обработан.

Задача 13. Удаление type code

Что сделали: Заменили switch по строковому типу на полиморфизм через интерфейс NotificationSender.

Ответ для защиты: Полиморфизм заменяет switch по строке. Каждое поведение инкапсулировано в классе. Добавление нового типа не требует изменения клиента. Полиморфизм оправдан, когда меняется поведение, а не просто данные конфигурации.

Задача 14. Наследование взрывает комбинации

Что сделали: Разделили формат курса и политику оплаты на две независимые оси (CourseFormat и PaymentPolicy). Собрали их через композицию в классе Course.

Ответ для защиты: Две независимые оси дают 9 комбинаций, что ведёт к комбинаторному взрыву. Композиция позволяет собирать любые комбинации из 3+3 классов. Добавление нового формата или оплаты требует создания всего одного класса.

Задача 15. Расширяемый движок правил

Что сделали: Создали sealed RuleResult (Ok / Fail), интерфейс Rule, композиции AllOf и AnyOf. Сервис принимает любое правило и обрабатывает результат через switch.

Ответ для защиты: Наследование уместно для RuleResult (неизменяемая структура данных с жёстким набором вариантов). Композиция используется в AllOf и AnyOf (хранят массив правил). sealed гарантирует, что switch покрывает все варианты. Сервис не знает о конкретных правилах — он зависит только от интерфейса Rule.

Заключение

В ходе лабораторной работы были изучены и закреплены на практике механизмы полиморфизма Java: динамическая диспетчеризация, перегрузка и переопределение методов, интерфейсы и абстрактные классы, композиция и делегирование, контракты equals/hashCode, sealed-иерархии и расширяемый движок правил.

На задачах 1–5 разобраны базовые механизмы полиморфизма: один контракт с несколькими реализациями, два этапа выбора метода (overload

компилятором, override JVM), полиморфный массив без if и instanceof, шаблонный метод с final-алгоритмом и абстрактной точкой расширения, а также критерии выбора между интерфейсом и абстрактным классом.
На задачах 6–10 изучены проектные приёмы: роль @Override при защите от случайной перегрузки, замена наследования композицией через декораторы, соблюдение принципа подстановки Лисков за счёт разделения интерфейсов, паттерн «Стратегия» для политик допуска и разрешение конфликта default-методов через Interface.super.method().
На задачах 11–15 рассмотрены более сложные темы: симметрия equals в иерархии наследования и её решение через композицию, sealed-иерархия результата с исчерпывающим switch, удаление type code через полиморфизм, устранение комбинаторного взрыва разделением независимых осей и построение расширяемого движка правил на композиции AllOf и AnyOf.
Все 15 заданий реализованы в виде отдельных папок, каждая программа компилируется и запускается без ошибок. Работа выполнена на оценку «5».
Резюме: пройден путь от базового полиморфизма (Задача 1) до проектирования расширяемых систем с sealed-иерархиями и композицией (Задача 15). Все задания демонстрируют, что клиент зависит только от абстракций (интерфейсов), а конкретные реализации можно менять и добавлять, не трогая существующий код.